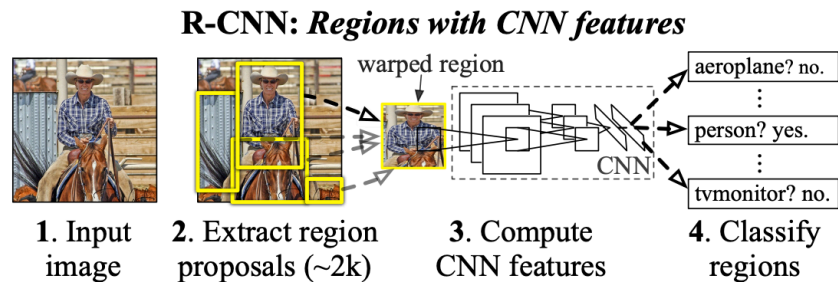


faster R-CNN

R-CNN(Region-based Convolutional Neural Networks) 모델

Object Detection 모델은 객체의 위치를 찾고(Localization), class를 분류(Classification)하는 작업을 진행합니다. R-CNN 모델은 이를 순차적으로 진행하는 대표적인 **2 stage detector**로, 딥러닝을 적용한 최초의 Object Detection 모델입니다. R-CNN 모델은 객체 탐지(detection) 혹은 추론(inference) 시, 다음과 같은 순서에 따라 동작합니다.



R-CNN 모델의 동작 순서

1. **Selective search** 알고리즘을 통해 객체가 있을 법한 위치인 후보 영역(region proposal)을 2000개 추출하여, 각각을 227x227 크기로 **warp**시켜줍니다.
2. Warp된 모든 region proposal을 **Fine tune된 AlexNet**에 입력하여 2000x4096 크기의 feature vector를 추출합니다.
3. 추출된 feature vector를 **linear SVM** 모델과 **Bounding box regressor** 모델에 입력하여 각각 confidence score와 조정된 bounding box 좌표를 얻습니다.
4. 마지막으로 **Non maximum suppression** 알고리즘을 적용하여 최소한의, 최적의 bounding box를 출력합니다.

1. Region proposal by Selective search

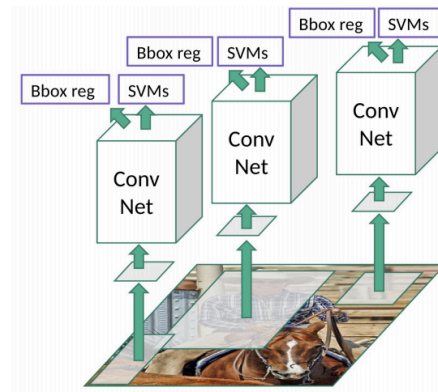


Selective search

R-CNN 모델은 구체적인 객체의 위치를 추정하기 앞서 **Selective search** 알고리즘을 통해 객체가 있을 법한 위치인 **후보 영역(Region proposal)**을 추출합니다. Selective search 알고리즘은 색상, 무늬, 명암 등의 다양한 기준으로 픽셀을 grouping하고, 점차 통합시켜 객체가 있을 법한 위치를 bounding box 형태로 추천합니다. 단일 이미지에서 2000개의 후보 영역을 추출한 뒤, CNN 모델에 입력하기 위해 227x227 크기로 **warp**(=resize)시켜줍니다.

- **Input** : Single image
- **Process** : Region proposal by Selective search & warp
- **Output** : 227×227 sized 2000 region proposals

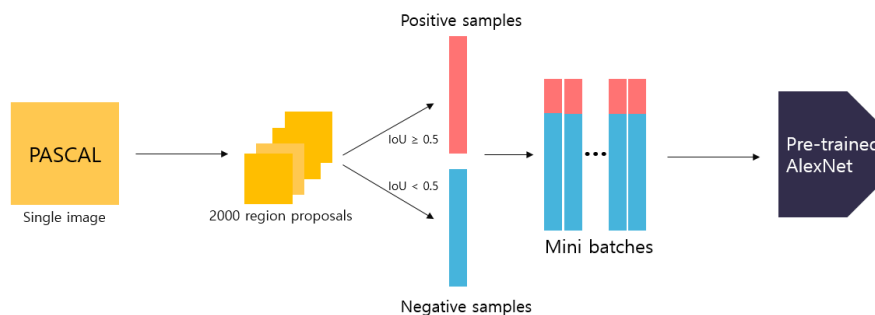
2. Feature extraction by Fine tuned AlexNet



Fine tuning AlexNet

다음으로 2000개의 후보영역을 **Fine tune된 AlexNet**에 입력하여 $2000(=\text{후보 영역의 수}) \times 4096(=\text{feature vector의 차원})$ 크기의 **feature vector**를 추출합니다. 논문의 저자는 객체 탐지시, 특정 도메인에 맞는 class를 예측하기 위해서 기존의 ImageNet 데이터셋을 통해 pre-trained된 CNN 모델을 도메인에 맞게 fine tune하는 방식을 제안합니다. Fine tune된 모델을 사용하면 도메인에 맞게, 보다 적합한 feature vector를 추출하는 것이 가능해집니다.

Fine tuning pre-trained AlexNet



Fine tuning pre-trained AlexNet

R-CNN 모델이 추론 시 입력으로 들어온 region proposal은 객체를 포함할 수도 있으며, 배경을 포함하고 있을 수도 있습니다. 따라서 fine tune 시 예측하려는 객체의 수가 N개라고 할 때 배경을 포함하여 **(N+1)개의 class를 예측하도록 모델을 설계(output unit=N+1)**해야 하며, **객체와 배경을 모두 포함한 학습 데이터를 구성**해야 합니다. 이를 위해 PASCAL VOC 데이터셋에 Selective search 알고리즘을 적용하여 객체와 배경이 모두 포함된 후보 영역을 추출하여 학습 데이터로 사용합니다. *Selective search 알고리즘은 R-CNN 모델의 추론 시에도, AlexNet을 fine tune할 때도 사용하는 셈입니다.*

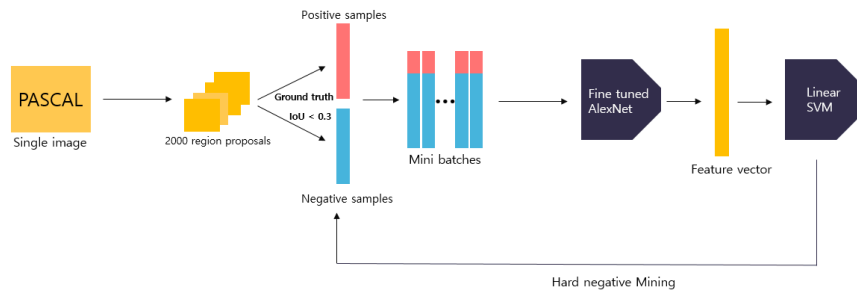
먼저 PASCAL VOC 데이터셋에 Selective search 알고리즘을 적용하여 후보 영역을 추출하고, 후보 영역(=bounding box)과 ground truth box와의 IoU 값을 구합니다. IoU 값이 0.5 이상인 경우 positive sample(=객체)로, 0.5 미만인 경우에는 negative sample(=배경)로 저장합니다. 그리고 positive sample = 32, negative sample = 96 로 mini batch(=128)을 구성하여 pre-trained된 AlexNet에 입력하여 학습을 진행합니다. 위의 과정을 통해 fine tune된 AlexNet을 사용하여 R-CNN 모델은 추론 시, feature vector를 추출합니다.

- **Input** : 227×227 sized 2000 region proposals
- **Process** : Feature extraction by fine tuned AlexNet
- **Output** : 2000×4096 sized feature vector

3. Classification by linear SVM

linear SVM(Support Vector Machine) 모델은 2000×4096 feature vector를 입력 받아 **class를 예측하고 confidence score**를 반환합니다. 이 때 linear SVM 모델은 특정 class에 해당하는지 여부만을 판단하는 **이진 분류기(binary classifier)**입니다. 따라서 N개의 class를 예측한다고 할 때, 배경을 포함한 **(N+1)개의 독립적인 linear SVM 모델을 학습**시켜야 합니다. 이제 단일 linear SVM 모델을 학습시키는 과정을 살펴보도록 하겠습니다.

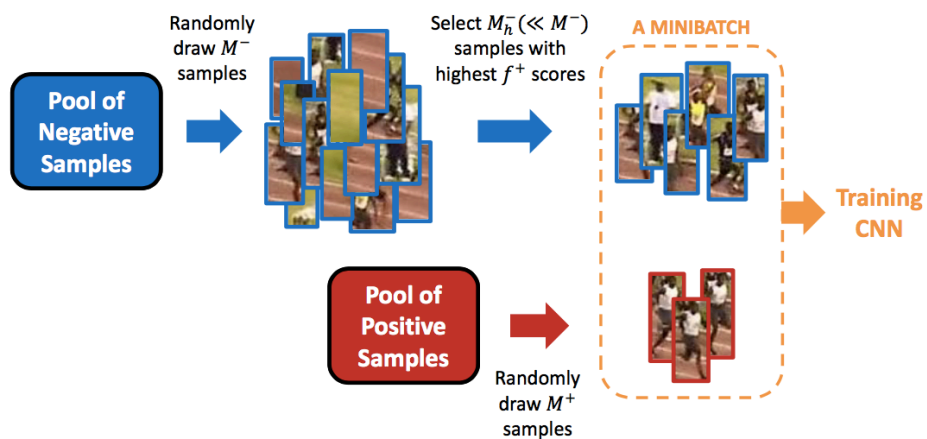
Training linear SVM using fine tuned AlexNet

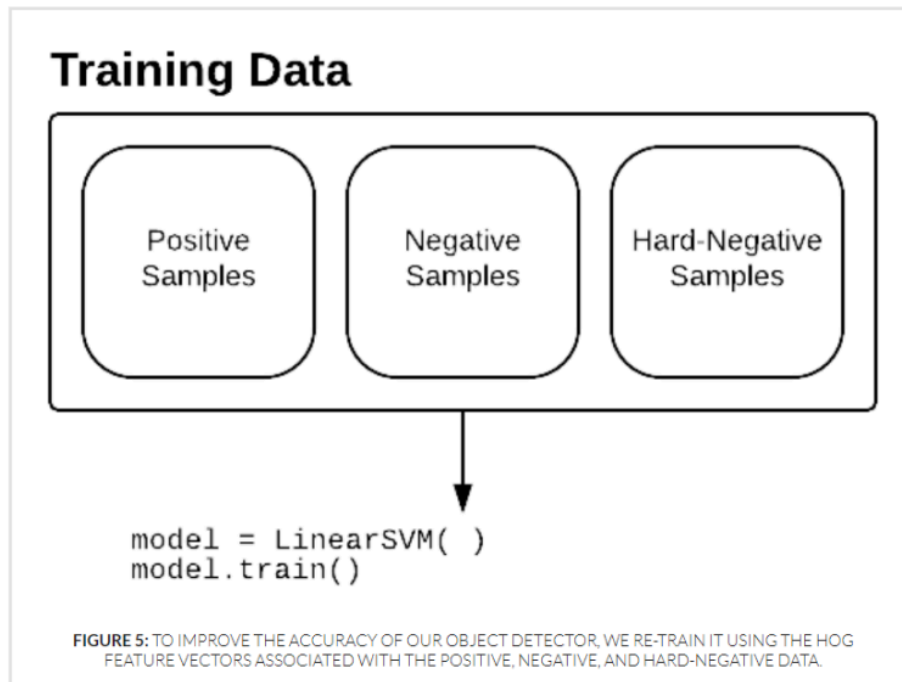


Training linear SVM using fine tuned AlexNet

먼저 객체와 배경을 모두 학습하기 위해 PASCAL VOC 데이터셋에 Selective search 알고리즘을 적용하여 region proposals를 추출합니다. AlexNet 모델을 fine tune할 때와는 다르게 **오직 ground truth box만을 positive sample로, IoU 값이 0.3 미만인 예측 bounding box를 negative sample로** 저장합니다. IoU 값이 0.3 이상인 bounding box는 무시합니다. positive sample = 32, negative sample = 96 이 되도록 mini batch(=128)을 구성한 뒤 fine tuned AlexNet에 입력하여 feature vector를 추출하고, 이를 linear SVM에 입력하여 학습시킵니다. 이 때 하나의 linear SVM 모델은 특정 class에 해당하는지 여부를 학습하기 때문에 **output unit = 2** 입니다. 학습이 한 차례 끝난 후, **hard negative mining** 기법을 적용하여 재학습시킵니다.

Hard Negative Mining





Hard negative mining

Hard negative mining은 모델이 예측에 실패하는 **어려운(hard)** sample들을 모으는 기법으로, hard negative mining을 통해 수집된 데이터를 활용하여 모델을 보다 강건하게 학습시키는 것이 가능해집니다. 예를 들어 이미지에서 사람의 안면의 위치를 탐지하는 모델을 학습시킨다고 할 때, 사람의 안면은 positive sample이며, 그외의 배경은 negative sample입니다. 이 때 모델이 배경이라고 예측했으며, 실제로 배경인 bounding box는 True Negative에 해당하는 sample입니다. 반면에 모델이 안면이라고 예측했지만, 실제로 배경인 경우는 **False Positive sample**에 해당합니다.

모델은 주로 False Positive라고 예측하는 오류를 주로 범합니다. 이는 객체 탐지 시, 객체의 위치에 해당하는 positive sample보다 배경에 해당하는 negative sample이 훨씬 많은 **클래스 불균형(class imbalance)**으로 인해 발생합니다. 이러한 문제를 해결하기 위해 모델이 잘못 판단한 False Positive sample을 학습 과정에서 추가하여 재학습하면 모델은 보다 강건해지며, False Positive라고 판단하는 오류가 줄어듭니다.

다음으로 논문의 저자가 linear SVM 모델 학습 시 생기는 여러 의문점들에 대해 답한 내용을 살펴보도록 하겠습니다.

Questions

Q1) 왜 AlexNet을 fine tune할 때와 linear SVM을 학습시킬 때와 양성/음성의 정의가 서로 다른가요?

논문의 저자는 AlexNet을 fine tune할 시, 모델이 **overfitting**되지 않도록 많은 데이터가 필요하다고 말합니다. IoU 값이 0.5 이상인 경우를 positive sample로 정의할 경우, 오직 ground truth box만을 positive sample로 정의할 때보다 30배 이상 많은 학습 데이터를 얻을 수 있다고 합니다.

Q2) 왜 AlexNet 마지막 layer에 softmax 함수를 추가하는 대신 별도의 linear SVM 모델을 학습시켰나요?

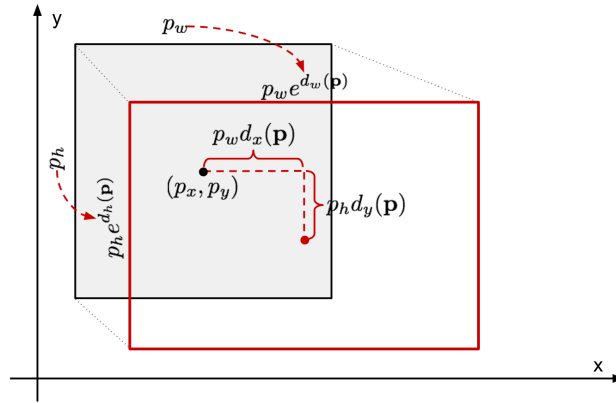
Softmax 함수를 사용하여 class를 분류할 경우, mAP 수치가 54.2%에서 50.9%로 떨어진다고 합니다. 이러한 이유는 위에서 언급한 모델에 따른 서로 다른 양성/음성 정의에서 기인합니다. AlexNet을 fine tune할 때, 많은 데이터를 포함하는 과정에서 상대적으로 정확하지 않은 예측 bounding box(IoU 값이 0.5~1 사이, 논문의 저자는 *jitered example*이라고 언급함)를 positive sample에 포함시키게 됩니다. 이로 인해 상대적으로 엄밀한 학습 데이터를 사용하는 linear SVM 모델에 비해 성능 하락하게 됩니다. 논문의 저자는 만약 적절한 방식으로 AlexNet을 fine tune한다면 linear SVM 모델을 사용할 때와 비슷한 성능을 보일 것이라고 전망합니다.

학습된 linear SVM에 2000x4096 크기의 feature vector를 입력하면 class와 confidence score를 반환합니다.

- **Input** : 2000×4096 sized feature vector
- **Process** : class prediction by linear SVM
- **Output** : 2000 classes and confidence scores

4. Detailed localization by Bounding box Regressor

Selective search 알고리즘을 통해 얻은 객체의 위치는 다소 부정확할 수 있습니다. 이러한 문제를 해결하기 위해 bounding box의 좌표를 변환하여 객체의 위치를 세밀하게 조정해주는 **Bounding box regressor** 모델이 있습니다.



Bounding box regressor

$$\begin{aligned} \hat{g}_x &= p_w d_x(p) + p_x, \hat{g}_y = p_h d_y(p) + p_y, \hat{g}_w = p_w \exp(d_w(p)), \hat{g}_h = p_h \exp(d_h(p)) \\ \hat{g}_x &= p_w d_x(p) + p_x, \hat{g}_y = p_h d_y(p) + p_y, \hat{g}_w = p_w \exp(d_w(p)), \hat{g}_h = p_h \exp(d_h(p)) \\ t_x &= (g_x - p_x) / p_w, t_y = (g_y - p_y) / p_h, t_w = \log(g_w / p_w), t_h = \log(g_h / p_h) \\ t_x &= (g_x - p_x) / p_w, t_y = (g_y - p_y) / p_h, t_w = \log(g_w / p_w), t_h = \log(g_h / p_h) \\ L_{reg} &= \sum_i \{x, y, w, h\} (t_i - d_i(p))^2 + \lambda \|w\|^2 \\ L_{reg} &= \sum_i \{x, y, w, h\} (t_i - d_i(p))^2 + \lambda \|w\|^2 \end{aligned}$$

위의 그림에서 회색 box는 Selective search 알고리즘에 의해 예측된 bounding box이며, 빨간 테두리 box는 ground truth box입니다. **Bounding box regressor**는 예측한 bounding box의 좌표 $p = (p_x, p_y, p_w, p_h)$ (center X, center Y, width, height)가 주어졌을 때, ground truth box의 좌표 $g = (g_x, g_y, g_w, g_h)$ 로 변환되도록 하는 **Scale invariant Transformation**을 학습합니다. $i \in \{x, y, w, h\}$ 라고 할 때 위의 변수는 다음과 같습니다.

$\hat{g}_i$: 예측한 bounding box p가 주어졌을 때, Bounding box regressor 모델이 변환한 결과

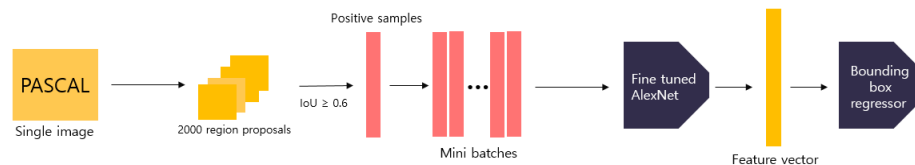
t_i : Bounding box regressor 모델이 학습하고자 하는 목표(target)

$d_i(P)$: Bounding box regressor 모델의 학습 대상

L_{reg} : Bounding box regressor 모델의 loss function으로 **SSE(Sum of Squared Error)**. $\lambda = 1000$

즉, Bounding box regressor 모델은 $d_i(P)$ 가 t_i 가 되도록 L_{reg} 를 통해 학습시킵니다.

Training Bounding box regressor using fine tuned AlexNet



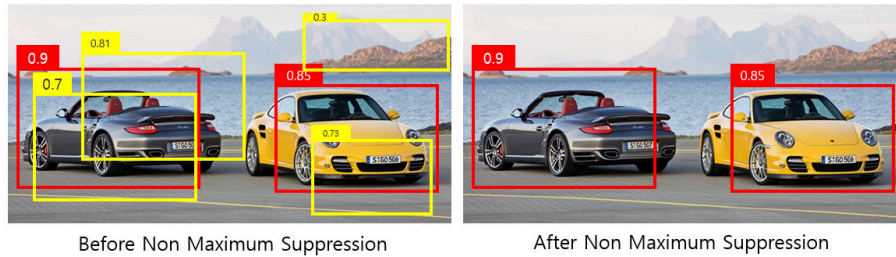
Training Bounding box regressor using fine tuned AlexNet

PASCAL 데이터셋에 Selective search 알고리즘을 적용하여 얻은 region proposals를 학습 데이터로 사용합니다. 이 때 별도의 negative sample은 정의하지 않고 IoU 값이 0.6 이상인 sample을 positive sample로 정의합니다. IoU 값이 지나치게 작거나 겹쳐진 영역이 없는 경우, 모델을 통해 학습시키기 어렵기 때문입니다. positive sample을 fine tuned된 AlexNet에 입력하여 얻은 feature vector를 Bounding box regressor에 입력하여 학습시킵니다. 추론 시 Bounding box regressor는 feature vector를 입력 받아 조정된 bounding box 좌표값(output unit=4)을 반환합니다.

- **Input** : 2000×4096 sized feature vector
- **Process** : bounding box coordinates transformation by Bounding box regressor
- **Output** : 2000 bounding box coordinates

5. Non maximum Suppression

linear SVM 모델과 Bounding box regressor 모델을 통해 얻은 2000개의 bounding box를 전부 다 표시할 경우 하나의 객체에 대해 지나치게 많은 bounding box가 겹칠 수 있습니다. 이로 인해 객체 탐지의 정확도가 떨어질 수 있습니다. 이러한 문제를 해결하기 감지된 bounding box 중에서 비슷한 위치에 있는 box를 제거하고 가장 적합한 box를 선택하는 **Non maximum suppression** 알고리즘을 적용합니다. Non maximum suppression 알고리즘은 아래와 같은 순서에 따라 진행됩니다.



Non maximum suppression

1. bounding box별로 지정한 confidence score threshold 이하의 box를 제거합니다.

- confidence score threshold = 0.5 로 지정합니다.
- 위의 그림에서 confidence score가 0.3인 box를 제거합니다.

2. 남은 bounding box를 confidence score에 따라 내림차순으로 정렬합니다. 그 다음 confidence score가 높은 순의 bounding box부터 다른 box와의 IoU 값을 조사하여 IoU threshold 이상인 box를 모두 제거합니다.

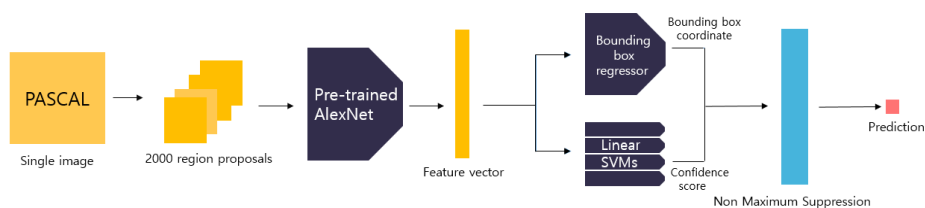
- IoU threshold = 0.4 로 지정합니다.
- confidence score에 따라 내림차순으로 box를 정렬합니다. [0.9, 0.85, 0.81, 0.73, 0.7]
- confidence score가 0.9인 box와 나머지 box와의 IoU 값 조사합니다. {0.85 : 0, 0.81 : 0.44, 0.73 : 0, 0.7 : 0.67}
- IoU threshold 이상인 confidence score가 0.81, 0.7인 box 제거합니다. [0.9, 0.85, 0.73]
- 남은 box에 대하여 위의 과정 반복합니다.

3. 남아있는 box만 선택

- 남은 box는 confidence score가 0.9, 0.85인 box 입니다.

Non maximum suppression 알고리즘은 confidence score threshold가 높을 수록, iou threshold가 낮을수록 많은 box가 제거됩니다. 위의 과정을 통해 겹쳐진 box를 제거하고 최소한의 최적의 bounding box를 반환합니다.

- **Input** : 2000 bounding boxes
- **Process** : removing unnecessary boxes with Non maximum suppression
- **Output** : optimal bounding boxes with classes

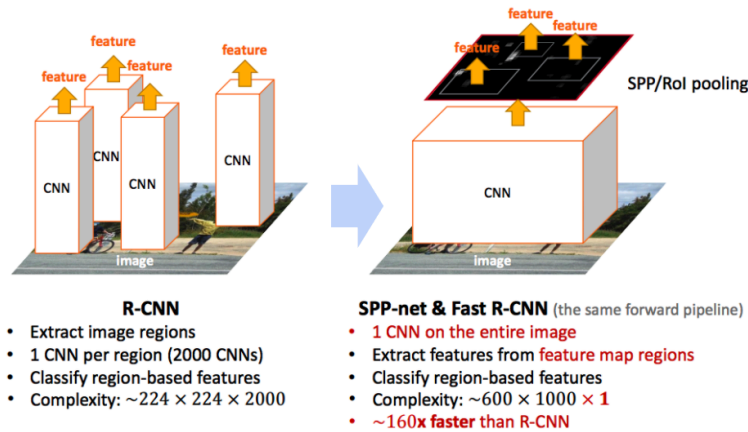


Object detection by R-CNN

지금까지 R-CNN 모델의 학습 방법과 추론에 대해 살펴보았습니다. R-CNN 모델은 동시대 객체 탐지 모델에 비해 성능이 매우 뛰어났으며, Object detection을 수행하기 위해 딥러닝을 최초로 적용했다는 점에서 의의가 있습니다. 하지만 Selective search를 사용하여 이미지의 한 장당 2000개의 region proposal을 추출하다보니, 학습 및 추론 속도가 매우 느리다는 단점이 있습니다. 또한 Fine tuned AlexNet, linear SVM, Bounding box regressor 라는 3가지 모델을 사용하다 보니 전체 구조와 학습 과정이 복잡하다는 문제가 있습니다.

Fast R-CNN 모델

Preview



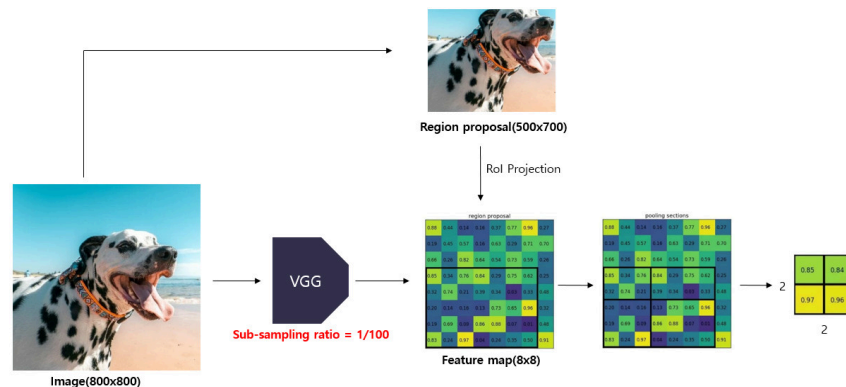
R-CNN vs Fast R-CNN

R-CNN 모델은 2000장의 region proposals를 CNN 모델에 입력시켜 각각에 대하여 독립적으로 학습시켜 많은 시간이 소요됩니다. Fast R-CNN은 이러한 문제를 개선하여 **단 1장의 이미지를 입력받으며**, region proposals의 크기를 warp시킬 필요 없이 **Roi(Region of Interest) pooling**을 통해 고정된 크기의 feature vector를 fully connected layer(이하 fc layer)에 전달합니다. 또한 **multi-task loss**를 사용하여 모델을 개별적으로 학습시킬 필요 없이 한 번에 학습시킵니다. 이를 통해 학습 및 detection 시간이 크게 감소하였습니다. 자세한 학습 과정을 살펴보기에 앞서 Fast R-CNN에 도입된 핵심 아이디어들을 살펴보도록 하겠습니다.

Main Ideas

1. Roi(Region of Interest) Pooling

Roi(Region of Interest) pooling은 feature map에서 region proposals에 해당하는 **관심 영역(Region of Interest)**을 지정한 크기의 grid로 나눈 후 max pooling을 수행하는 방법입니다. 각 channel별로 독립적으로 수행하며, 이 같은 방법을 통해 **고정된 크기의 feature map을 출력하는 것이 가능합니다**. 구체적인 동작 원리를 살펴보도록 하겠습니다(파란색 글씨는 위의 예제 해설입니다).



Roi pooling

1) 먼저 원본 이미지를 CNN 모델에 통과시켜 feature map을 얻습니다.

- **800x800** 크기의 이미지를 VGG 모델에 입력하여 **8x8** 크기의 feature map을 얻습니다.

- 이 때 **sub-sampling ratio = 1/100**이라고 할 수 있습니다(여기서 말하는 subsampling은 pooling을 거치는 과정을 의미).

2) 그리고 동시에 원본 이미지에 대하여 Selective search 알고리즘을 적용하여 region proposals를 얻습니다.

- 원본 이미지에 Selective search 알고리즘을 적용하여 **500x700** 크기의 region proposal을 얻습니다.

3) 이제 feature map에서 각 region proposals에 해당하는 영역을 추출합니다. 이 과정은 **Roi Projection**을 통해 가능합니다.

Selective search를 통해 얻은 region proposals는 sub-sampling 과정을 거치지 않은 반면, 원본 이미지의 feature map은 sub-sampling 과정을 여러 번 거쳐 크기가 작아졌습니다. **작아진 feature map에서 region proposals이 encode(표현)하고 있는 부분을 찾기 위해 작아진 feature map에 맞게 region proposals를 투영해주는 과정이 필요합니다**. 이는 region proposal의 크기와 중심 좌표를 **sub sampling ratio**에 맞게 변경시켜줌으로써 가능합니다.

- Region proposal의 중심점 좌표, width, height와 sub-sampling ratio를 활용하여 feature map으로 투영시켜줍니다.

- feature map에서 region proposal에 해당하는 **5x7** 영역을 추출합니다.

4) 추출한 RoI feature map을 지정한 **sub-window의 크기**에 맞게 grid로 나눠줍니다.

- 추출한 5x7 크기의 영역을 지정한 **2x2** 크기에 맞게 grid를 나눠줍니다.

5) grid의 각 셀에 대하여 max pooling을 수행하여 고정된 크기의 feature map을 얻습니다.

- 각 grid 셀마다 max pooling을 수행하여 **2x2** 크기의 feature map을 얻습니다.

이처럼 미리 지정한 크기의 sub-window에서 max pooling을 수행하다보니 **region proposal의 크기가 서로 달라도 고정된 크기의 feature map**을 얻을 수 있습니다.

2. Multi-task loss

Fast R-CNN 모델에서는 feature vector를 **multi-task loss**를 사용하여 Classifier와 Bounding box regressor를 동시에 학습시킵니다. 각각의 RoI(=region proposal)에 대하여 multi task loss를 사용하여 학습시킵니다. 이처럼 두 모델을 한번에 학습시키기 때문에, R-CNN 모델과 같이 **각 모델을 독립적으로 학습시켜야 하는 번거로움이 없다는 장점**이 있습니다. multi-task loss는 아래와 같습니다.

$$L(p, u, tu, v) = Lcls(p, u) + \lambda[u \geq 1]Lloc(tu, v) \quad L(p, u, tu, v) = Lcls(p, u) + \lambda[u \geq 1]Lloc(tu, v)$$

$p = (p_0, \dots, p_k)$: (K+1)개의 class score

u : ground truth class score

$tu = (tux, tuy, tuw, tuh)$: 예측한 bounding box 좌표를 조정하는 값

$v = (vx, vy, vw, vh)$: 실제 bounding box의 좌표값

$Lcls(p, u) = -\log p_u$: classification loss(Log loss)

$Lloc(tu, v) = \sum_{i \in \{x, y, w, h\}} smoothL1(tui - vi)$: regression loss(Smooth L1 loss)

$smoothL1(tui - vi) = \begin{cases} 0.5 \times 2, & \text{if } |x| < 1 \\ |x| - 0.5, & \text{otherwise} \end{cases}$

λ : 두 loss 사이의 가중치를 조정하는 balancing hyperparameter

- K개의 class를 분류한다고 할 때, 배경을 포함한 (K+1)개의 class에 대하여 Classifier를 학습시켜줘야 합니다.
- u는 positive sample인 경우 1, negative sample인 경우 0으로 설정되는 **index parameter**입니다.

u

- **L1 loss**는 R-CNN, SPPnets에서 사용한 L2 loss에 비해 outlier에 덜 민감하다는 장점이 있습니다.
- $\lambda=1$ 로 사용합니다.

λ

- multi task loss는 0.8~1.1% mAP를 상승시키는 효과가 있다고 합니다.

3. Hierarchical Sampling

R-CNN 모델은 학습 시 region proposal이 서로 다른 이미지에서 추출되고, 이로 인해 학습 시 연산을 공유할 수 없다는 단점이 있습니다. 논문의 저자는 학습 시 **feature sharing**을 가능하게 하는 **Hierarchical sampling** 방법을 제시합니다. SGD mini-batch를 구성할 때 N개의 이미지를 sampling하고, 총 R개의 region proposal을 사용한다고 할 때, 각 이미지로부터 R/N개의 region proposals를 sampling하는 방법입니다. 이를 통해 같은 이미지에서 추출된 region proposals끼리는 forward, backward propagation 시, **연산과 메모리를 공유할 수 있습니다**.

논문에서는 학습 시, N=2, R=128로 설정하여, 서로 다른 2장의 이미지에서 각각 64개의 region proposals를 sampling하여 mini-batch를 구성합니다. 각 이미지의 region proposals 중 25%(=16장)는 ground truth와의 IoU 값이 0.5 이상인 sample을 추출하고, 나머지(75%, 48장)에 대해서는 IoU 값이 0.1~0.5 사이의 sample을 추출합니다. **전자의 경우 positive sample로, 위에서 정의한 multi-task loss의 u=1**이며, 후자는 **u=0**인 경우라고 할 수 있습니다.

4. Truncated SVD

Fast R-CNN 모델은 detection 시, RoI를 처리할 때 fc layer에서 많은 시간을 잡아먹습니다. 논문에서는 detection 시간을 감소시키기 위해 **Truncated SVD(Singular Vector Decomposition)**을 통해 fc layer를 압축하는 방법을 제시합니다. SVD에 대한 설명은 [귀퉁이 서재님의 블로그](#)를 참고했습니다.

$$A = U \Sigma V^T$$

Full SVD

행렬 AA 를 $m \times m \times m$ 크기인 UU , $m \times n \times n$ 크기인 $\Sigma\Sigma$, $n \times n \times n$ 크기인 VTV^T 로 특이값 분해(SVD)하는 것을 **Full SVD(Singular Vector Decomposition)**라고 합니다. 하지만 실제로 이처럼 Full SVD를 하는 경우는 드물며, Truncated SVD와 같이 분해된 행렬 중 일부분만을 활용하는 reduced SVD를 일반적으로 많이 사용한다고 합니다.

$$A' = U_t \Sigma_t V_t^T$$

Truncated SVD

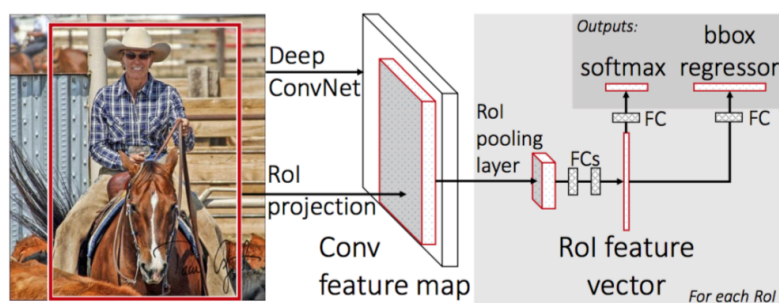
Truncated SVD는 $\Sigma\Sigma$ 의 비대각 부분과 대각 원소 중 특이값이 0인 부분을 모두 제거하고, 제거된 $\Sigma\Sigma$ 에 대응되는 UU , VV 원소도 함께 제거하여 차원을 줄인 형태입니다. $U_t U_t^T$ 의 크기는 $m \times t \times m$ 이며, $\Sigma_t \Sigma_t^T$ 의 크기는 $t \times t \times t$, 그리고 $V_t V_t^T$ 의 크기는 $t \times n \times n$ 입니다. 이렇게 하면 행렬 AA 를 상당히 근사하는 것이 가능합니다.

$$W \approx U \Sigma_t V_t^T W \approx U \Sigma_t V_t^T$$

fc layer의 가중치 행렬이 $W(=u \times v)W(=u \times v)$ 라고 할 때, Truncated SVD를 통해 위와 같이 근사하는 것이 가능합니다. 이를 통해 파라미터 수를 $u \times v \times u \times v$ 에서 $t(u+v)t(u+v)$ 로 감소시키는 것이 가능합니다. Truncated SVD를 fc layer의 가중치 행렬 WW 에 적용하면, fc layer는 두 개의 fc layer로 나뉘게 됩니다. 첫 번째 fc layer는 $\Sigma_t V_t^T \Sigma_t V_t^T$ 가중치 행렬, 두 번째 fc layer는 UU 가중치 행렬입니다. 이를 통해 네트워크를 효율적으로 압축하는 것이 가능하며, 논문의 저자는 **Truncated SVD를 통해 detection 시간이 30% 정도 감소**되었다고 말합니다.

Training Fast R-CNN

이제 본격적으로 Fast R-CNN 모델을 학습시키는 과정을 살펴보도록 하겠습니다. 하나의 이미지가 입력되었을 때를 가정하고 전체 학습 과정을 살펴보도록 하겠습니다.



Fast R-CNN architecture

1) Initializing pre-trained network

feature map을 추출하기 위해 **VGG16** 모델을 사용합니다. 먼저 네트워크를 detection task에 맞게 변형시켜주는 과정이 필요합니다.

1. VGG16 모델의 마지막 max pooling layer를 **RoI pooling layer**로 대체해줍니다. 이 때 RoI pooling을 통해 출력되는 feature map의 크기인 H, W 는 후속 fc layer와 호환 가능하도록 크기인 **7x7**로 설정해줍니다.
2. 네트워크의 마지막 fc layer를 2개의 fc layer로 대체합니다. 첫 번째 fc layer는 K 개의 class와 배경을 포함한 **(K+1)개의 output unit**을 가지는 **Classifier**이며, 두 번째 fc layer는 각 class별로 bounding box의 좌표를 조정하여 **(K+1) * 4개의 output unit**을 가지는 **bounding box regressor**입니다.



VGG16

3. conv layer3까지의 가중치값은 **고정(freeze)**시켜주고, 이후 layer(conv layer4~ fc layer3)까지의 가중치값이 학습될 수 있도록 **fine tuning**해줍니다. 논문의 저자는 fc layer만 fine tuning했을 때보다 conv layer까지 포함시켜 학습시켰을 때 더 좋은 성능을 보였다고 합니다.

4. 네트워크가 원본 이미지와 selective search 알고리즘을 통해 추출된 region proposals 집합을 입력으로 받을 수 있도록 변환시켜 줍니다.

2) region proposal by Selective search

먼저 원본 이미지에 대하여 Selective search 알고리즘을 적용하여 미리 region proposals를 추출합니다.

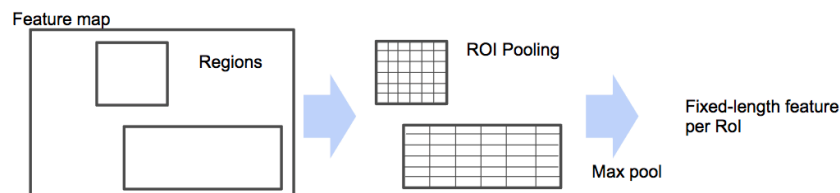
- **Input** : image
- **Process** : Selective search
- **Output** : 2000 region proposals

3) Feature extraction(~layer13 pre-pooling) by VGG16

VGG16 모델에 $224 \times 224 \times 3$ 크기의 원본 이미지를 입력하고, layer13까지의 feature map을 추출합니다. RoI pooling 과정은 바로 다음 단계에서 살펴보고, 이번 단계에서는 VGG16을 통해 추출된 feature map까지만 짚고 넘어가도록 하겠습니다. 마지막 pooling을 수행하기 전에 14×14 크기의 feature map 512개가 출력됩니다.

- **Input** : $224 \times 224 \times 3$ sized image
- **Process** : feature extraction by VGG16
- **Output** : $14 \times 14 \times 512$ feature maps

4) Max pooling by RoI pooling

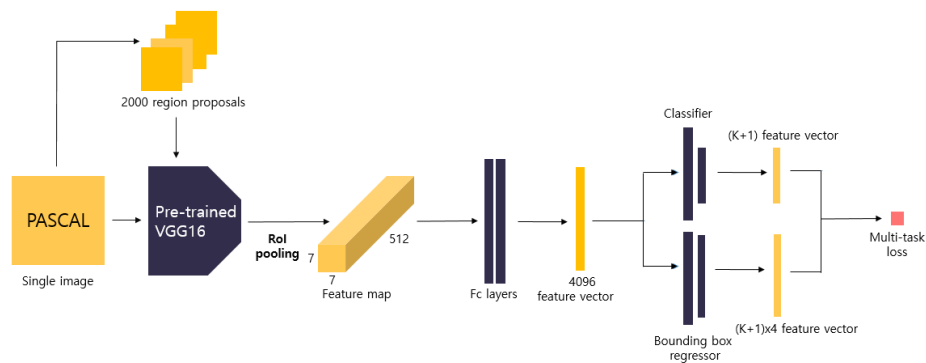


RoI pooling

region proposals를 layer13을 통해 출력된 feature map에 대하여 RoI projection을 진행한 후, **RoI pooling**을 수행합니다. 앞서 언급했듯이, RoI pooling layer는 VGG16의 마지막 pooling layer를 대체한 것입니다. 이 과정을 거쳐 고정된 7×7 크기의 feature map을 추출합니다.

- **Input** : 14×14 sized 512 feature maps, 2000 region proposals
- **Process** : RoI pooling
- **Output** : $7 \times 7 \times 512$ feature maps

5) Feature vector extraction by Fc layers



Training Fast R-CNN

다음으로 region proposal별로 $7 \times 7 \times 512 (=25088)$ 의 feature map을 flatten한 후 fc layer에 입력하여 fc layer를 통해 4096 크기의 feature vector를 얻습니다.

- **Input** : $7 \times 7 \times 512$ sized feature map
- **Process** : feature extraction by fc layers
- **Output** : 4096 sized feature vector

6) Class prediction by Classifier

4096 크기의 feature vector를 K개의 class와 배경을 포함하여 (K+1)개의 output unit을 가진 fc layer에 입력합니다. 하나의 이미지에서 하나의 region proposal에 대한 class prediction을 출력합니다.

- **Input** : 4096 sized feature vector
- **Process** : class prediction by Classifier
- **Output** : (K+1) sized vector(class score)

7) Detailed localization by Bounding box regressor

4096 크기의 feature vector를 class별로 bounding box의 좌표를 예측하도록 (K+1) x 4개의 output unit을 가진 fc layer에 입력합니다. 하나의 이미지에서 하나의 region proposal에 대한 class별로 조정된 bounding box 좌표값을 출력합니다.

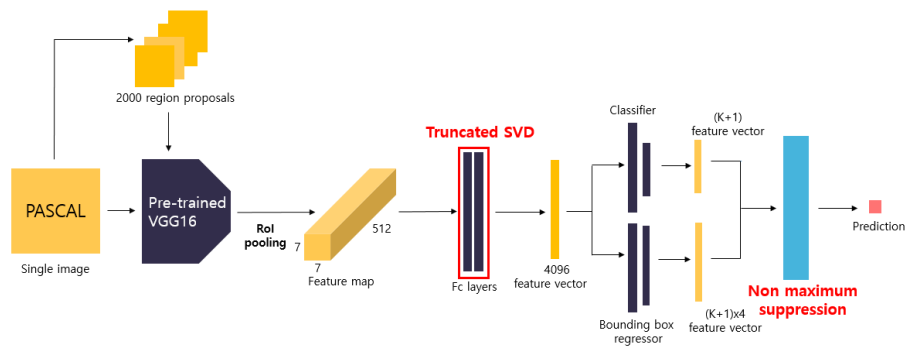
- **Input** : 4096 sized feature vector
- **Process** : Detailed localization by Bounding box regressor
- **Output** : (K+1) x 4 sized vector

8) Train Classifier and Bounding box regressor by Multi-task loss

Multi-task loss를 사용하여 하나의 region proposal에 대한 Classifier와 Bounding box regressor의 loss를 반환합니다. 이후 Backpropagation을 통해 두 모델(Classifier, Bounding box regressor)을 한 번에 학습시킵니다.

- **Input** : (K+1) sized vector(class score), (K+1) x 4 sized vector
- **Process** : calculate loss by Multi-task loss function
- **Output** : loss(Log loss + Smooth L1 loss)

Detection Fast R-CNN



Object detection by Fast R-CNN

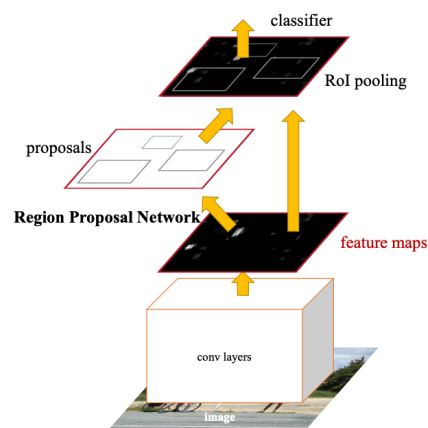
다음으로 실제 detection 시, Fast R-CNN 모델의 동작을 살펴보도록 하겠습니다. Detection 시 동작 순서는 학습 과정과 크게 다르지 않습니다. 하지만 4096 크기의 feature vector를 출력하는 fc layer에 **Truncated SVD**를 적용한다는 점에서 차이가 있습니다. 또한 예측한 bounding box에 대하여 **Non maximum suppression** 알고리즘이 추가되어 최적의 bounding box만을 출력하게 됩니다.

지금까지 Fast R-CNN 모델에 대해 살펴보았습니다. Fast R-CNN 모델은 R-CNN 모델보다 학습 속도가 9배 이상 빠르며, detection 시, 이미지 한 장당 0.3초(region proposals 추출 시간 포함)이 걸립니다. PASCAL VOC 2012 Challenge에서 mAP 값이 66%를 보이면서 detection 성능 역시 R-CNN 모델에 비해 향상된 모습을 보입니다. 이외에도 multi-task loss를 사용하여 single stage로 여러 모델을 학습시킬 수 있다는 점과, 학습으로 인해 네트워크의 가중치값을 Backpropagation을 통해 update할 수 있다는 장점이 있습니다.

Faster R-CNN 모델

기존 Fast R-CNN 모델은 여전히 Selective search 알고리즘을 통해 region proposals 추출하기 때문에 학습 및 detection 속도를 향상시키는데 한계가 있습니다. 또한 detection을 위한 과정을 end-to-end로 수행하지 못한다는 문제가 있습니다. 이러한 문제를 해결하여 속도와 모델의 완성도 측면에서 더 좋은 모습을 보인 **Faster R-CNN 모델**을 살펴보도록 하겠습니다.

Preview



Faster R-CNN

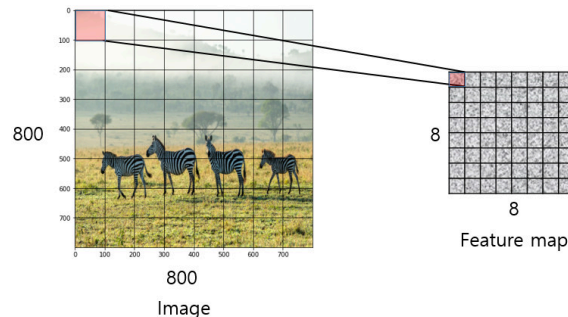
후보 영역 추출을 위해 사용되는 Selective search 알고리즘은 CPU 상에서 동작하고 이로 인해 네트워크에서 병목현상이 발생하게 됩니다. Faster R-CNN은 이러한 문제를 해결하고자 **후보 영역 추출 작업을 수행하는 네트워크인 Region Proposal Network(이하 RPN)**를 도입합니다. RPN은 region proposals를 보다 정교하게 추출하기 위해 다양한 크기와 가로세로비를 가지는 bounding box인 **Anchor box**를 도입합니다. **Faster R-CNN 모델을 간략하게 보면 RPN과 Fast R-CNN 모델이 합쳐졌다고 볼 수 있습니다.** RPN에서 region proposals를 추출하고 이를 Fast R-CNN 네트워크에 전달하여 객체의 class와 위치를 예측합니다. 이를 통해 모델의 전체 과정이 GPU 상에서 동작하여 병목 현상이 발생하지 않으며, end-to-end로 네트워크를 학습시키는 것이 가능해집니다. 전체적인 동작 순서는 아래와 같습니다.

- 1) 원본 이미지를 pre-trained된 CNN 모델에 입력하여 feature map을 얻습니다.
- 2) feature map은 RPN에 전달되어 적절한 region proposals를 산출합니다.

- 3) region proposals와 1) 과정에서 얻은 feature map을 통해 RoI pooling을 수행하여 고정된 크기의 feature map을 얻습니다.
- 4) Fast R-CNN 모델에 고정된 크기의 feature map을 입력하여 Classification과 Bounding box regression을 수행합니다.

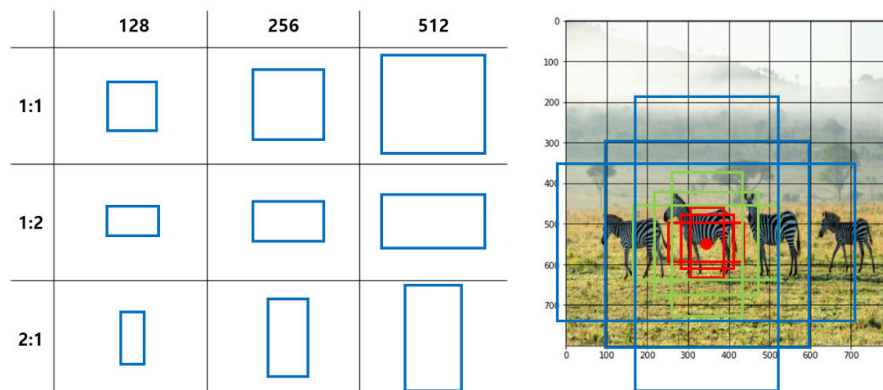
Main Ideas

1. Anchor box



Dense Sampling

Selective search를 통해 region proposal을 추출하지 않을 경우, 원본 이미지를 일정 간격의 grid로 나눠 각 grid cell을 bounding box로 간주하여 feature map에 encode하는 **Dense Sampling** 방식을 사용합니다. 이같은 경우 sub-sampling ratio를 기준으로 grid를 나누게 됩니다. 가령 원본 이미지의 크기가 800x800이며, sub-sampling ratio가 1/100이라고 할 때, CNN 모델에 입력시켜 얻은 최종 feature map의 크기는 8x8(800x1/100)가 됩니다. 여기서 feature map의 각 cell은 원본 이미지의 100x100만큼의 영역에 대한 정보를 함축하고 있다고 할 수 있습니다. 원본 이미지에서는 8x8개만큼의 bounding box가 생성된다고 볼 수 있습니다.



Anchor boxes

하지만 이처럼 고정된 크기(fixed size)의 bounding box를 사용할 경우, 다양한 크기의 객체를 포착하지 못할 수 있다는 문제가 있습니다. 본 논문에서는 이러한 문제를 해결하고자 지정한 위치에 사전에 정의한 서로 다른 크기(scale)와 가로세로비(aspect ratio)를 가지는 bounding box인 **Anchor box**를 생성하여 다양한 크기의 객체를 포착하는 방법을 제시합니다. 논문에서 3가지 scale([128, 256, 512])과 3가지 aspect ratio([1:1, 1:2, 2:1])를 가지는 총 9개의 서로 다른 anchor box를 사전에 정의(pre-define)합니다.

$$w \times h = s^2 w \times h = s^2$$

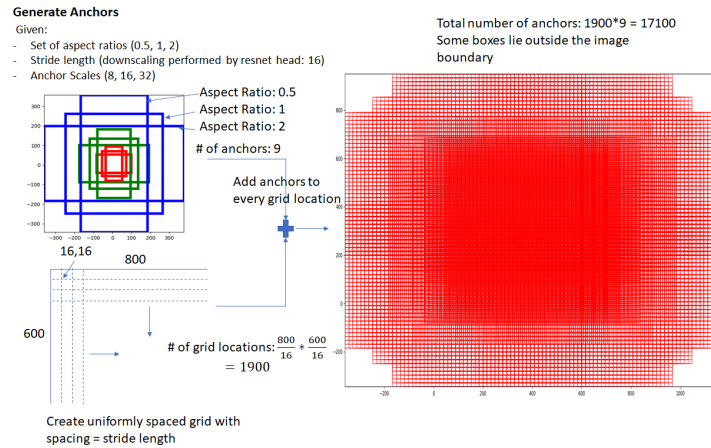
$$w = 12 \times h = 12 \times h$$

$$12 \times h^2 = s^2 12 \times h^2 = s^2$$

$$h = \sqrt{s^2 12} = \sqrt{12} s$$

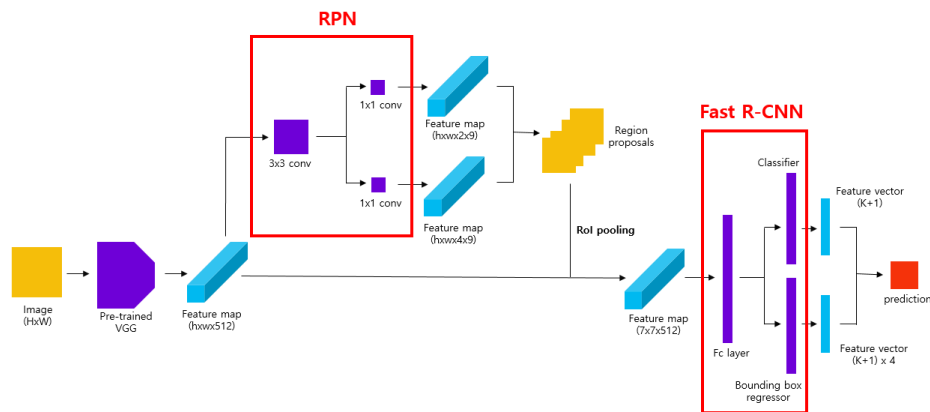
$$w = \sqrt{12} s^2 w = 2 s^2$$

여기서 scale은 anchor box의 width(=ww), height(=hh)의 길이를, aspect ratio는 width, height의 길이의 비율을 의미합니다. 여기서 aspect ratio에 따른 width, height의 길이는 **aspect ratio가 1:1일 때의 anchor box의 넓이를 유지한 채** 구합니다. 예를 들어 scale이 ss이며, aspect ratio가 1:1일 때의 anchor box의 넓이는 $s^2(=s \times s)s^2(=s \times s)$ 입니다. 여기서 aspect ratio가 1:2, 즉 height가 width의 2배일 때 위와 같은 수식에 따라 width, height를 구합니다. aspect ratio가 2:1인 경우에도 마찬가지로 구합니다.



anchor box는 원본 이미지의 각 grid cell의 중심을 기준으로 생성합니다. 원본 이미지에서 sub-sampling ratio를 기준으로 anchor box를 생성하는 기준점인 anchor를 고정합니다. 이 anchor를 기준으로 사전에 정의한 anchor box 9개를 생성합니다. 위의 그림에서 원본 이미지의 크기는 600x800이며, sub-sampling ratio=1/16 입니다. 이 때 anchor가 생성되는 수는 $1900(=600/16 \times 800/16)$ 이며, anchor box는 총 $17100(=1900 \times 9)$ 개가 생성됩니다. 이같은 방식을 사용할 경우, 기존에 고정된 크기의 bounding box를 사용할 때보다 9배 많은 bounding box를 생성하며, 보다 다양한 크기의 객체를 포착하는 것이 가능합니다.

2. RPN(Region Proposal Network)



Region proposal Network

RPN은 원본 이미지에서 region proposals를 추출하는 네트워크입니다. 원본 이미지에서 anchor box를 생성하면 수많은 region proposals가 만들어집니다. RPN은 region proposals에 대하여 class score를 매기고, bounding box coefficient를 출력하는 기능을 합니다. RPN의 전체적인 동작 과정을 간략하게 살펴보겠습니다(파란색 글씨는 예시입니다).

1) 원본 이미지를 pre-trained된 VGG 모델에 입력하여 feature map을 얻습니다.

원본 이미지의 크기가 800x800이며, sub-sampling ratio가 1/100이라고 했을 때 8x8 크기의 feature map이 생성됩니다(channel 수는 512개 입니다).

2) 위에서 얻은 feature map에 대하여 3x3 conv 연산을 적용합니다. 이때 feature map의 크기가 유지될 수 있도록 padding을 추가합니다.

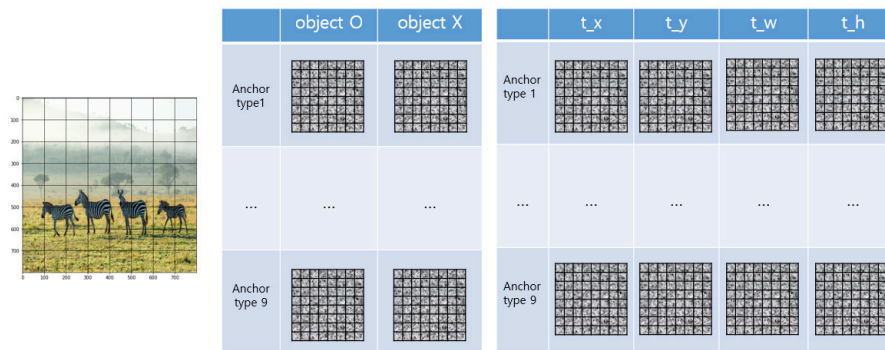
8x8x512 feature map에 대하여 3x3 연산을 적용하여 8x8x512개의 feature map이 출력됩니다.

3) class score를 매기기 위해서 feature map에 대하여 1x1 conv 연산을 적용합니다. 이 때 출력하는 feature map의 channel 수가 2x9가 되도록 설정합니다. RPN에서는 후보 영역이 어떤 class에 해당하는지까지 구체적인 분류를 하지 않고 객체가 포함되어 있는지 여부를 분류합니다. 또한 anchor box를 각 grid cell마다 9개가 되도록 설정했습니다. 따라서 **channel 수는 2(object 여부) x 9(anchor box 9개)**가 됩니다.

8x8x512 크기의 feature map을 입력받아 8x8x2x9크기의 feature map을 출력합니다.

4) bounding box regressor를 얻기 위해 feature map에 대하여 1x1 conv 연산을 적용합니다. 이 때 출력하는 feature map의 channel 수가 **4(bounding box regressor)x9(anchor box 9개)**가 되도록 설정합니다.

8×8×512 크기의 feature map을 입력받아 8×8×4×9크기의 feature map을 출력합니다.



RPN result

RPN의 출력 결과는 위와 같습니다. 좌측 표는 anchor box의 종류에 따라 객체 포함 여부를 나타낸 feature map이며, 우측 표는 anchor box의 종류에 따라 bounding box regressor를 나타낸 feature map입니다. 이를 통해 8x8 grid cell마다 9개의 anchor box가 생성되어 총 576(=8x8x9)개의 region proposals가 추출되며, feature map을 통해 각각에 대한 객체 포함 여부와 bounding box regressor를 파악할 수 있습니다. 이후 class score에 따라 상위 N개의 region proposals만을 추출하고, Non maximum suppression을 적용하여 최적의 region proposals만을 Fast R-CNN에 전달하게 되는데, 이 과정은 아래에서 구체적으로 살펴보도록 하겠습니다.

3. Multi-task loss

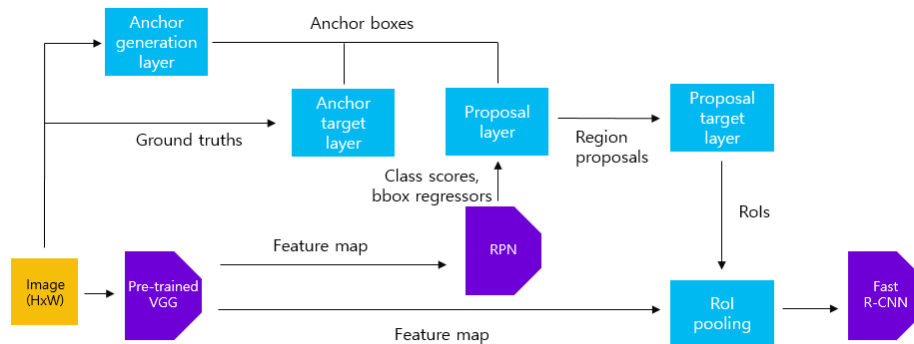
$$L(\{p_i\}, \{t_i\}) = 1/N_{cls} \sum_i L_{cls}(p_i, p^*_i) + \lambda 1/N_{reg} \sum_i p_i^* L_{reg}(t_i, t^*_i) \quad L(\{p_i\}, \{t_i\}) = 1/N_{cls} \sum_i L_{cls}(p_i, p^*_i) + \lambda 1/N_{reg} \sum_i p_i^* L_{reg}(t_i, t^*_i)$$

- i : mini-batch 내의 anchor의 index
 i
- p_i : anchor i 에 객체가 포함되어 있을 예측 확률
 p_i
 i
- p^*_i : anchor가 양성일 경우 1, 음성일 경우 0을 나타내는 index parameter
 p^*_i
- t_i : 예측 bounding box의 파라미터화된 좌표(coefficient)
 t_i
- t^*_i : ground truth box의 파라미터화된 좌표
 t^*_i
- L_{cls} : Loss loss
 L_{cls}
- L_{reg} : Smooth L1 loss
 L_{reg}
- N_{cls} : mini-batch의 크기(논문에서는 256으로 지정)
 N_{cls}
- N_{reg} : anchor 위치의 수
 N_{reg}
- λ : balancing parameter(default=10)
 λ

RPN과 Fast R-CNN을 학습시키기 위해 Multi-task loss를 사용합니다. 하지만 RPN에서는 객체의 존재 여부만을 분류하는 반면, Fast R-CNN에서는 배경을 포함한 class를 분류한다는 점에서 차이가 있습니다.

Training Faster R-CNN

이제 본격적으로 Faster R-CNN 모델의 학습 과정을 살펴보도록 하겠습니다. 흔히 $\text{Faster R-CNN} = \text{RPN} + \text{Fast R-CNN}$ 이라고 단순히 설명합니다. 하지만 단순히 위의 수식으로 설명하기에는 모델 내부에서 처리해야 하는 다양한 작업들이 있어 코드를 살펴보면 상당히 복잡합니다. 대표적으로 anchor를 생성하고 처리하는 작업과 적절한 region proposals를 추출하는 작업이 있습니다. 저는 우선 Faster R-CNN 모델에서 처리하는 동작 과정을 분석한 후 논문에서 언급한 모델의 실제 학습 방법을 살펴보도록 하겠습니다. 전체적인 동작 과정은 [Ankur Mohan님의 블로그](#)를 참고했습니다.



Training Faster R-CNN

1) feature extraction by pre-trained VGG16

pre-trained된 VGG16 모델에 $800 \times 800 \times 3$ 크기의 원본 이미지를 입력하여 $50 \times 50 \times 512$ 크기의 feature map을 얻습니다. 여기서 sub-sampling ratio는 1/16입니다.

- **Input** : $800 \times 800 \times 3$ sized image
- **Process** : feature extraction by pre-trained VGG16
- **Output** : $50 \times 50 \times 512$ sized feature map

2) Generate Anchors by Anchor generation layer

region proposals를 추출하기에 앞서 **원본 이미지에 대하여 anchor box를 생성**하는 과정이 필요합니다. 원본 이미지의 크기에 sub-sampling ratio를 곱한만큼의 grid cell이 생성되며, 이를 기준으로 각 grid cell마다 9개의 anchor box를 생성합니다. 즉, 원본 이미지에 $50 \times 50 (= 800 \times 1/16 \times 800 \times 1/16)$ 개의 grid cell이 생성되고, 각 grid cell마다 9개의 anchor box를 생성하므로 총 $22500 (= 50 \times 50 \times 9)$ 개의 anchor box가 생성됩니다.

- **Input** : $800 \times 800 \times 3$ sized image
- **Process** : generate anchors
- **Output** : $22500 (= 50 \times 50 \times 9)$ anchor boxes

3) Class scores and Bounding box regressor by RPN

RPN은 VGG16으로부터 feature map을 입력받아 anchor에 대한 **class score, bounding box regressor**를 반환하는 역할을 합니다. 동작 과정은 위에서 설명한 부분을 참고하시면 될 것 같습니다.

- **Input** : $50 \times 50 \times 512$ sized feature map
- **Process** : Region proposal by RPN
- **Output** : class scores($50 \times 50 \times 2 \times 9$ sized feature map) and bounding box regressors($50 \times 50 \times 4 \times 9$ sized feature map)

4) Region proposal by Proposal layer

Proposal layer에서는 2)번 과정에서 생성된 anchor boxes와 RPN에서 반환한 class scores와 bounding box regressor를 사용하여 **region proposals**를 추출하는 작업을 수행합니다. 먼저 Non maximum suppression을 적용하여 부적절한 객체를 제거한 후, class score 상위 N개의 anchor box를 추출합니다. 이후 regression coefficients를 anchor box에 적용하여 anchor box가 객체의 위치를 더 잘 detect하도록 조정합니다.

- **Input**
 - $22500 (= 50 \times 50 \times 9)$ anchor boxes
 - class scores($50 \times 50 \times 2 \times 9$ sized feature map) and bounding box regressors($50 \times 50 \times 4 \times 9$ sized feature map)

- **Process** : region proposal by proposal layer
- **Output** : top-N ranked region proposals

5) Select anchors for training RPN by Anchor target layer

Anchor target layer의 목표는 **RPN이 학습하는데 사용할 수 있는 anchor를 선택**하는 것입니다. 먼저 2)번 과정에서 생성한 anchor box 중에서 원본 이미지의 경계를 벗어나지 않는 anchor box를 선택합니다. 그 다음 positive/negative 데이터를 sampling해줍니다. 여기 positive sample은 객체가 존재하는 foreground, negative sample은 객체가 존재하지 않는 background를 의미합니다.

전체 anchor box 중에서 1) ground truth box와 가장 큰 IoU 값을 가지는 경우 2) ground truth box와의 IoU 값이 0.7 이상인 경우에 해당하는 box를 positive sample로 선정합니다. 반면 ground truth box와의 IoU 값이 0.3 이하인 경우에는 negative sample로 선정합니다. IoU 값이 0.3~0.7인 anchor box는 무시합니다. 이러한 과정을 통해 RPN을 학습시키는데 사용할 데이터셋을 구성하게 됩니다.

- **Input** : anchor boxes, ground truth boxes
- **Process** : select anchors for training RPN
- **Output** : positive/negative samples with target regression coefficients

6) Select anchors for training Fast R-CNN by Proposal Target layer

Proposal target layer의 목표는 proposal layer에서 나온 region proposals 중에서 **Fast R-CNN 모델을 학습시키기 위한 유용한 sample을 선택**하는 것입니다. 여기서 선택된 region proposals는 1)번 과정을 통해 출력된 feature map에 RoI pooling을 수행하게 됩니다. 먼저 region proposals와 ground truth box와의 IoU를 계산하여 0.5 이상일 경우 positive, 0.1~0.5 사이일 경우 negative sample로 label됩니다.

- **Input** : top-N ranked region proposals, ground truth boxes
- **Process** : select region proposals for training Fast R-CNN
- **Output** : positive/negative samples with target regression coefficients

7) Max pooling by RoI pooling

원본 이미지를 VGG16 모델에 입력하여 얻은 feature map과 6) 과정을 통해 얻은 sample을 사용하여 RoI pooling을 수행합니다. 이를 통해 고정된 크기의 feature map이 출력됩니다. RoI pooling에 대한 자세한 설명은 [Fast R-CNN 논문 리뷰](#)를 참고하시기 바랍니다.

- **Input**
 - 50×50×512 sized feature map
 - positive/negative samples with target regression coefficients
- **Process** : RoI pooling
- **Output** : 7×7×512 sized feature map

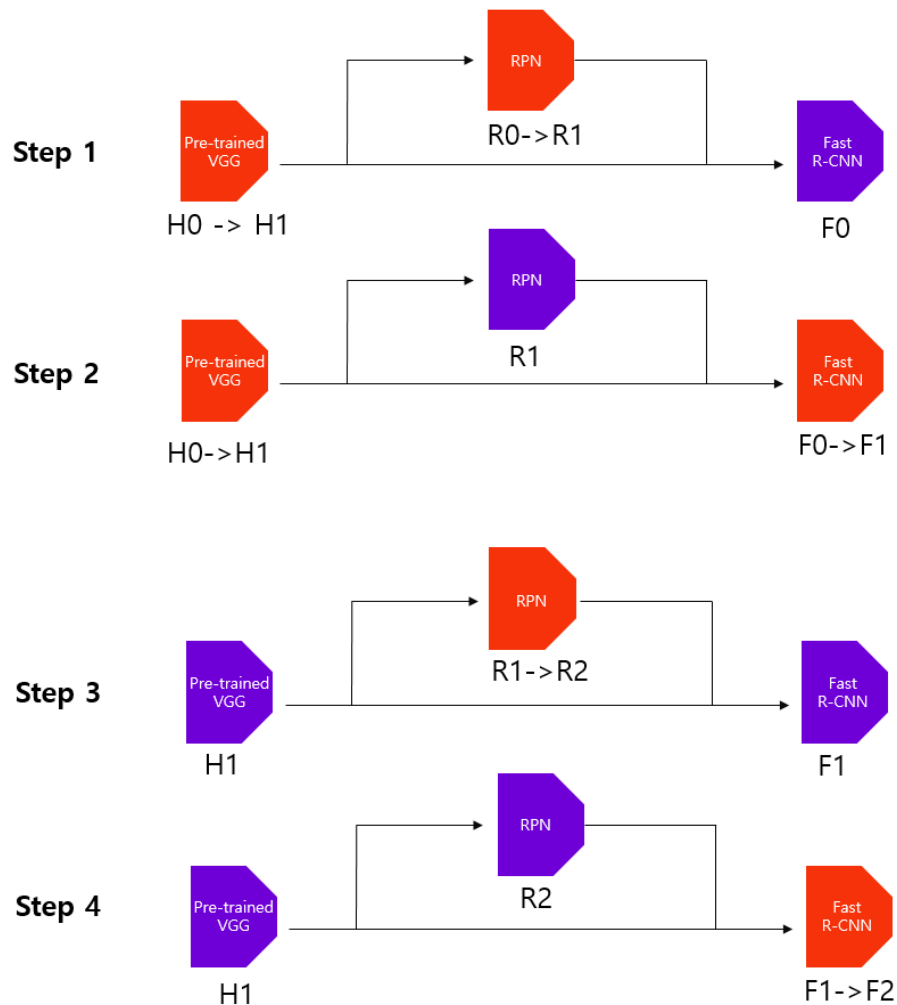
8) Train Fast R-CNN by Multi-task loss

나머지 과정은 Fast R-CNN 모델의 동작 순서와 동일합니다. 입력받은 feature map을 fc layer에 입력하여 4096 크기의 feature vector를 얻습니다. 이후 feature vector를 Classifier와 Bounding box regressor에 입력하여 (class의 수가 K라고 할 때) 각각 (K+1), (K+1) × 4 크기의 feature vector를 출력합니다. 출력된 결과를 사용하여 Multi-task loss를 통해 Fast R-CNN 모델을 학습시킵니다. 자세한 내용은 [Fast R-CNN 논문 리뷰](#)를 참고하시기 바랍니다.

- **Input** : 7×7×512 sized feature map
- **Process**
 - feature extraction by fc layer
 - classification by Classifier
 - bounding box regression by Bounding box regressor
 - Train Fast R-CNN by Multi-task loss
- **Output** : loss(Loss loss + Smooth L1 loss)

Alternating Training

논문의 저자는 Faster R-CNN 모델을 학습시키기 위해 RPN과 Fast R-CNN을 번갈아가며 학습시키는 **Alternating Training** 방법을 사용합니다. 학습 과정은 아래와 같습니다.



Alternating Training

1) 먼저 Anchor generation layer에서 생성된 anchor box와 원본 이미지의 ground truth box를 사용하여 Anchor target layer에서 RPN을 학습시킬 positive/negative 데이터셋을 구성합니다. 이를 활용하여 **RPN을 학습**시킵니다. 이 과정에서 pre-trained된 VGG16 역시 학습됩니다.

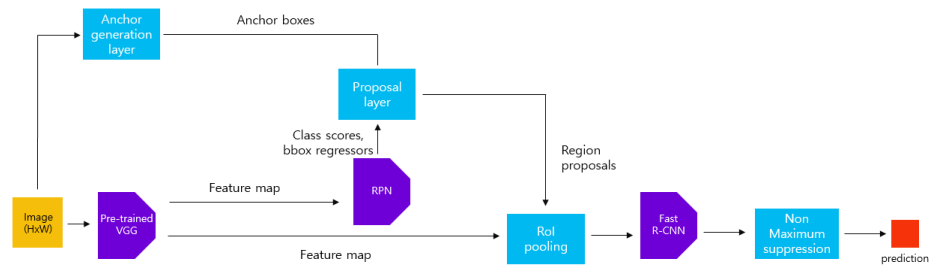
2) Anchor generation layer에서 생성한 anchor box와 학습된 RPN에 원본 이미지를 입력하여 얻은 feature maps를 사용하여 proposals layer에서 region proposals를 추출합니다. 이를 Proposal target layer에 전달하여 Fast R-CNN 모델을 학습시킬 positive/negative 데이터셋을 구성합니다. 이를 활용하여 **Fast R-CNN을 학습**시킵니다. 이 때 pre-trained된 VGG16 역시 학습됩니다.

3) 앞서 학습시킨 RPN과 Fast R-CNN에서 **RPN에 해당하는 부분만 학습(fine tune)**시킵니다. 세부적인 학습 과정은 1)과 같습니다. 이 과정에서 두 네트워크끼리 공유하는 convolutional layer, 즉 **pre-trained된 VGG16은 고정(freeze)**합니다.

4) 학습시킨 RPN(3)번 과정을 활용하여 추출한 region proposals를 활용하여 **Fast R-CNN을 학습(fine tune)**시킵니다. 이 때 **RPN과 pre-trained된 VGG16은 고정(freeze)**합니다.

쉽게 생각하면 RPN과 Fast R-CNN을 번갈아가며 학습시키면서 공유된 convolutional layer를 사용한다고 보면 됩니다. 하지만 실제 학습 절차가 상당히 복잡하여 이후 두 네트워크를 병합하여 학습시키는 **Approximate Joint Training** 방법으로 대체된다고 합니다.

Detection



Faster R-CNN detection

실제 detection(=inference) 시에는 Anchor target layer와 Proposal target layer는 사용되지 않습니다. 두 layer 모두 네트워크를 학습시키기 위한 데이터셋을 구성하는데 사용되기 때문입니다. Fast R-CNN은 Proposal layer에서 추출한 region proposals를 활용하여 detection을 수행합니다. 그리고 최종적으로 얻은 predicted box에 **Non maximum suppression**을 적용하여 최적의 bounding box만을 결과로 출력합니다.

지금까지 Faster R-CNN 모델에 대해 살펴보았습니다. Faster R-CNN 모델은 PASCAL VOC 2012 데이터셋에서 mAP 값이 75.9를 보이면서 Fast R-CNN 모델보다 더 높은 detection 성능을 보였습니다. 또한 Fast R-CNN 모델이 0.5fps(frame pre second)인 반면 Faster R-CNN 모델은 17fps를 보이며, 이미지 처리 속도 면에서 발전한 결과를 보였습니다. 또한 feature extraction에 사용하는 convolutional layer의 feature를 공유하면서 end-to-end로 학습시키는 것이 가능해졌습니다. 하지만 논문의 저자는 detection 속도에 대해 "near real-time"이라고 언급하며, 실시간 detection에는 한계가 있음을 인정했습니다.